Lezione 2 - Il Props Object

Introduzione

Nella scorsa lezione abbiamo visto cosa sono i componenti funzionali in React e le loro caratteristiche, ora vediamo come passare dei parametri ai componenti

Il Props Object

In React, **le props** (abbreviazione di *properties*, cioè *proprietà*) rappresentano il meccanismo principale attraverso cui un componente **riceve informazioni dall'esterno**.

Ogni **componente React** possiede un oggetto chiamato `props`, **che contiene tutti i dati passati al componente dal suo elemento genitore.**
Possiamo quindi dire che:

> ✅ **Le *props* sono un oggetto che mantiene le informazioni relative a un componente.**

Esempio concreto:

Consideriamo un semplice elemento HTML:

React JSX

```
<button type="submit" value="Submit"> Submit </button>
```

In questo caso, il tag `<button>` riceve due informazioni tramite attributi:

- `type="submit"`
- `value="Submit"`

Questi attributi determinano **come** il bottone si comporta: ad esempio, il valore del `type` decide se il pulsante invierà una form, reimposterà i campi o agirà come un semplice bottone.

Allo stesso modo, in React possiamo **trasmettere informazioni personalizzate** ai nostri componenti, in modo che si comportino o si presentino diversamente a seconda dei dati ricevuti.

📋 Parallelismo tra Props e funzioni:

Le props in un componente React svolgono lo **stesso ruolo degli argomenti in una funzione JavaScript**.
Infatti, quando dichiariamo un componente come funzione, React passa automaticamente l'oggetto `props` come argomento:

React JSX

```
function Welcome(props) {
 return <h1>Ciao, {props.name}!</h1>;
}

```

E quando lo utilizziamo:

React JSX

```
<Welcome name="Marco" />

```

React, quindi crea un oggetto:

JavaScript

```
{name: "Marco"}
```

e lo passa al componente come parametro `props`.

Come accedere ai valori delle props:

Per accedere ai dati contenuti nelle props, si utilizza la **dot notation**, ovvero la notazione con il punto:

React JSX

```
props.name
```

In questo modo si recupera il valore associato alla proprietà `name` dell'oggetto `props`.

Consiglio

I parametri che passiamo ad un componente vengono chiamati per convenzione props, si consiglia di usare questa convenzione in modo da rendere il nostro codice il più leggibile possibile.

Passare le Props a un Componente

Per poter sfruttare i vantaggi offerti dalle **props**, è necessario **passare delle informazioni a un componente React**.

Se non passiamo alcuna proprietà, l'oggetto `props` all'interno del componente risulterà **vuoto**.
Ad esempio, se creiamo un componente `<PropsDisplayer />` in `App.jsx`, senza alcun attributo, questo riceverà un oggetto `props` privo di contenuti.

Come si passano le props

Per passare una props a un componente, utilizziamo la stessa sintassi degli attributi HTML.
Ecco un esempio semplice:

React JSX

```
@App.jsx
import React from 'react';
import Greeting from './Greeting.jsx'



function App(){
    return (
        <Greeting name = "Jamel" />
    )
}
export default App;



@Greeting.jsx
const Greeting = (props)=>{
    return <h1>{props.name}</h1>
```

```
}
export default Greeting;

```

In questo caso stiamo passando una props chiamata `name` con valore `"Jamel"`. Il componente `Greeting` creato nel file `Greeting.jsx` potrà poi accedere a questo valore tramite `props.name` se il parametro `props` verrà inserito in input al componente.

Un altro esempio

Supponiamo di avere un componente `SloganDisplay` e di voler passargli un messaggio.
Possiamo farlo allo stesso modo di `Greeting`:

React JSX

```

@App.jsx
import React from 'react';
import Greeting from './Greeting.jsx';
import SloganDisplay from './SloganDisplay.jsx';



function App(){
    return (
        <Greeting name = "Jamel" />
        <SloganDisplay message = "We're great!" />
    );
}
export default App;


@SloganDisplay.jsx
const SloganDisplay = (props)=>{
    return <h3>{props.name}</h3>
}
export default SloganDisplay;

```

Come possiamo notare, per passare un'informazione a un componente React servono **due elementi fondamentali**:

1. Un **nome** per la proprietà (in questo caso `message`) definito in App.jsx poichè è lì che il componente andrà renderizzato;
2. Un **valore** da associare a quella proprietà (in questo caso la stringa `"We're great!"`).

> 💡 *Nota:* il nome della props può essere qualsiasi, purché sia significativo per ciò che rappresenta.

Passare tipi di dati diversi dalle stringhe

Quando vogliamo passare un valore che **non è una stringa**, ad esempio numeri, array o oggetti, dobbiamo **racchiuderlo tra parentesi graffe** `{}`.

Ecco un esempio pratico:

React JSX

```jsx
@App.jsx
import React from 'react';
import Greeting from './Greeting.jsx';
import PropsDisplayer from './PropsDisplayer.jsx';


function App(){
    return(
        <Greeting myInfo = {["Astronaut", "Narek", 43]} />
        <PropsDisplayer
            name = "hello"
            int = {54}
            float = {3.14}
            array = {["hi", 54, 3.14, false]}
            object = {{name:"Marco", age: 27}}
        />
    )
}
export default App;
```

Importante:

Solo le **stringhe** possono essere passate senza le parentesi graffe.
Tutti gli altri tipi di dati (numeri, booleani, array, oggetti, espressioni) devono essere *wrappati* tra `{}`.

Visualizzare l'oggetto props:

Vediamo ora un esempio di componente che mostra il contenuto dell'oggetto `props` :

React JSX

```
@App.jsx
import React from 'react';
import Greeting from './Greeting.jsx';
import PropsDisplayer from './PropsDisplayer.jsx';


function App(){
    return(
        <Greeting myInfo = {["Astronaut", "Narek", 43]} />
        <PropsDisplayer
            name = "hello"
            int = {54}
            float = {3.14}
            array = {["hi", 54, 3.14, false]}
            object = {{name:"Marco", age: 27}}
        />
    )
}
export default App;




@PropsDisplayer.jsx
function PropsDisplayer(props) {
  const stringProps = JSON.stringify(props);
  return (
    <div>
      <h1>CHECK OUT MY PROPS OBJECT</h1>
      <h2>{stringProps}</h2>
    </div>
  );
}

export default PropsDisplayer;

```

Cosa succede qui:

1. `props` **come parametro:**
   Il componente riceve un argomento chiamato `props`, che contiene tutte le proprietà passate quando il componente è stato usato (es. `<PropsDisplayer name="Marco" />`).
2. `JSON.stringify(props)`**:**
   Converte l'oggetto `props` in una stringa JSON leggibile, così possiamo visualizzarlo nell'interfaccia.

> ⚠ **Attenzione:**
>
> - `JSON.stringify()` ignora le funzioni e i valori `undefined`;
> - se l'oggetto contiene riferimenti circolari, lancerà un errore;
> - per un output più leggibile si può usare `JSON.stringify(props, null, 2)` avvolto in un `<pre>`.

3. **Il return JSX:**
   Il componente restituisce del markup JSX con un titolo e il contenuto testuale dell'oggetto props.
   Le **parentesi graffe** in JSX (`{stringProps}`) indicano che il valore deve essere valutato come **espressione JavaScript**, non come testo statico.
   L'output quindi sarà:

```
CHECK OUT MY PROPS OBJECT
{"name":"hello","int":54,"float":3.14,"array":
["hi",54,3.14,false],"object":{"name":"marco","age":27}}
```

Renderizzare le Props di un Componente

Le **props** permettono di *personalizzare* un componente React, passando informazioni dall'esterno per modificarne il comportamento o il contenuto.
Nella sezione precedente abbiamo visto **come passare** i dati a un componente; ora vediamo **come utilizzarli e renderizzarli** all'interno di esso.

Uso delle props all'interno di un componente

Per assicurarsi che un componente possa accedere alle proprie props, è sufficiente dichiararle come parametro della funzione.

Esempio di base:

React JSX

```jsx
@App.jsx
import React from 'react';
import Button from './Button.jsx';

function App(){
    return(
        <Button displayText = "Cliccami!" />
    )
}
export default App;

@Button.jsx
const Button = (props) => {
    return <button>{props.displayText}</button>;
}
export default Button
```

In questo caso:

- `props` è l'oggetto che contiene tutte le informazioni passate al componente;
- per accedere a una proprietà specifica si utilizza la **dot notation**, cioè `props.nomeProprietà`.

Nell'esempio sopra, `props.displayText` rappresenta il testo da mostrare all'interno del bottone.
Nel file `App.jsx` infatti scriviamo `<Button displayText="Cliccami!" />`, il bottone renderizza "Cliccami!".

Alternativa: usare la destrutturazione delle props

Poiché le props sono un normale oggetto JavaScript, possiamo sfruttare la **destrutturazione** per estrarre solo le proprietà che ci servono, rendendo il codice più pulito e leggibile:

React JSX

```jsx
function Button({ displayText }) {
    return <button>{displayText}</button>;
}
```

Questa sintassi è del tutto equivalente alla precedente, ma evita di dover scrivere ogni volta `props.` davanti al nome della proprietà.
È una forma molto comune e consigliata in React moderno.

Esempio pratico:

Immaginiamo ora di avere un componente che rappresenta un prodotto:

React JSX

```
@App.jsx
import React from 'react';
import Product from './Product.jsx';

function App(){
    return(
        <Product name = "Apple McAir" price = {999€} rating = "4,5"/>
    )
}
export default App;



@Product.jsx
const Product = (props) =>{
    return (
        <div>
            <h1>{props.name}</h1>
            <h2>{props.price}</h2>
            <h3>{props.rating}</h3>
        </div>
    )
}

// con la destrutturizazione diventa :
@Product.jsx
const Product = ({name, price, rating}) =>{
    return (
        <div>
            <h1>{name}</h1>
            <h2>{price}</h2>
            <h3>{rating}</h3>
        </div>
```

```
34        )
35    }
```

In questo esempio:

- `name`, `price` e `rating` sono **attributi** passati al componente `Product` dal file `App.jsx`, ad esempio:

React JSX

```
1   <Product name="Laptop" price="999€" rating="4.5" />
```

- Il componente accede e mostra ciascun valore tramite `props.nomeProprietà`.

In sintesi

- Le props consentono di *trasmettere dati* ai componenti React.
- Per *usarle*, basta dichiararle come parametro della funzione e accedervi con la dot notation.
- In alternativa, la **destrutturazione** permette di estrarre solo le proprietà necessarie in modo più leggibile.
- Renderizzare le props significa visualizzare nell'interfaccia i dati passati, adattando così il comportamento o l'aspetto del componente in base a ciò che riceve.

Passare le Props da un Componente a un Altro

Finora abbiamo visto:

- come **passare le props** a un componente,
- e come **accedere e visualizzare** queste props all'interno del componente stesso.

Tuttavia, l'uso più comune delle props in React è trasmettere informazioni da un componente a un altro.
Questo è un concetto fondamentale nella logica di React.

Il flusso delle props : "dall'alto verso il basso"

Le props in React seguono un **flusso unidirezionale** (*one-way data flow*):

> **dal componente padre verso il componente figlio**.

Ciò significa che un **componente padre può passare dati ai propri figli, ma i figli non possono modificarli direttamente.**

Esempio:

React JSX

```
function App() {
    return <Product name="Apple Watch" price={399} rating="4.5/5.0" />;
}
```

In questo caso:

- `App` **è il componente padre;**
- `Product` **è il componente figlio;**
- **il padre passa tre props (** `name` **,** `price` **,** `rating` **) al figlio;**
- **il componente** `Product` **può leggere queste props, ma non modificarle**

Quindi le props sono immutabili:
Le props passate ai componenti figli sono immutabili, ovvero non possono essere cambiate direttamente dal figlio.
Se un componente necessita di valori diversi, **sarà il padre a dover passare nuove props aggiornate.**

Questo meccanismo è ciò che garantisce **ordine e prevedibilità** nel flusso dei dati di un'app React.

Esempio pratico : passare props da `App` a `Player`

Vediamo come mettere in pratica questi concetti:

1. **Il componente padre (** `App` **)** passerà i dati al componente figlio (`Player`):

React JSX

```
import React from 'react';
import Player from './Player.jsx';

function App(){
    return (
        <Player songName="Moonlight Sonata" artist="Beethoven" />
```

```
    )
}

export default App
```

2. **Il componente figlio (** `Player` **)** riceve le props come parametro e le utilizza:

React JSX

```
function Player(props) {
    return (
        <>
            <h1>{props.songName}</h1>
            <h2>{props.artist}</h2>
        </>
    );
}

export default Player

```

In questo modo:

- `App` **fornisce le informazioni (nome della canzone e artista);**
- `Player` **si limita a visualizzare tali informazioni, mantenendo la logica unidirezionale dei dati.**

Renderizzare UI differenti in base alle Props

Un altro potente utilizzo delle **props** è la possibilità di **modificare dinamicamente l'interfaccia utente** (*UI*) in base ai valori ricevuti.
In altre parole, possiamo far sì che un componente mostri contenuti diversi a seconda delle props che gli vengono passate.

Esempio: condizioni basate sulle props

Immaginiamo un componente `LoginMsg` che visualizzi un messaggio diverso in base alla password ricevuta come prop:

React JSX

```
const LoginMsg = (props)=> {
  if (props.password === 'a-tough-password') {
    return <h2>Sign In Successful.</h2>;
```

```
4     } else {
5       return <h2>Sign In Failed.</h2>;
6     }
7   }
8   export default LoginMsg
```

In questo esempio:

- **La prop `password` non viene mostrata a schermo, ma è usata per prendere una decisione logica.**
- Se il valore di `password` è `"a-tough-password"`, React renderizza il messaggio “ `Sign In Successful.` ”.
- In caso contrario, mostra “ `Sign In Failed.` ”.

> **Info**
>
> Questa tecnica — usare le props per decidere **cosa renderizzare** — **è molto comune e costituisce una delle basi della renderizzazione condizionale in React.**

Esempio pratico: gestire l’accesso con una prop booleana

Vediamo ora un esempio più concreto, in cui una prop booleana (`signedIn`) determina se mostrare un messaggio di benvenuto o un invito al login:

React JSX

```
1   const Greeting = (props) => {
2     if (props.signedIn === false) {
3       return <h1>Please Login.</h1>;
4     } else {
5       return (
6         <>
7           <h1>Welcome back, {props.name}!</h1>
8           <article>
9             React: una serie di sfortunate disabilità
10          </article>
11        </>
12      );
13    }
14  }
15  export default Greeting
```

Analizziamolo passo per passo:

- La prop `signedIn` viene **passata dal componente padre** (`App`) e può essere `true` o `false` .
- Il blocco `if...else` viene eseguito **prima del return JSX**, quindi il componente decide *cosa restituire* in base al valore di `signedIn` .
- Se `signedIn` è `false` , viene mostrato “Please Login.”
- Se invece `signedIn` è `true` , l’utente è considerato autenticato e viene accolto con un messaggio personalizzato.

In sintesi

- Le props possono essere usate non solo per **visualizzare** informazioni, ma anche per **controllare il flusso logico** del rendering.
- La **renderizzazione condizionale** permette di mostrare UI diverse a seconda delle props ricevute.
- Questa è una delle tecniche più diffuse per gestire **stati logici, accessi utente, preferenze e comportamenti dinamici** nei componenti React.

Le Props Figlie (props.children)

In React, ogni componente riceve un oggetto chiamato **props**, che contiene tutte le informazioni (o proprietà) passate al componente come attributi.
Tuttavia, esiste una **prop speciale** che React aggiunge automaticamente a ogni componente: `props.children` .

Cos’è `props.children`

La proprietà `children` rappresenta tutto ciò che si trova tra i tag di apertura e chiusura di un componente.
Fino a ora, abbiamo visto componenti “self-closing”, cioè scritti nella forma:

React JSX

```
<MyComponent />
```

Ma è anche possibile scriverli nella forma estesa:

React JSX

```
import React from 'react';
import MyComponent from './MyComponent.jsx';

function App(){
    return (
    <MyComponent>
          {/* Contenuto interno */}
    </MyComponent>

    );
}
export default App;
```

In questo caso, **il contenuto interno** (testo, elementi HTML, o altri componenti React) viene automaticamente passato come valore di `props.children`.

Perché usare le props figlie

Usare `props.children` ci permette di **rendere i componenti più flessibili e riutilizzabili**.
Separando il **contenitore** (il componente padre) dal **contenuto** (il componente figlio o i figli), possiamo creare strutture componibili, in cui i componenti possono essere facilmente **annidati** tra loro.

In pratica, è il meccanismo che consente a React di costruire interfacce modulari e dinamiche.

Esempio pratico: il componente `BigButton`

React JSX

```
const BigButton =(props) =>{
    console.log (props.children)
    return <button>I am a Big Button</button>
}
```

- `props` è un oggetto che contiene tutte le informazioni passate al componente, compresa la chiave speciale `children`.
- In questo caso, il componente stampa in console ciò che riceve come contenuto figlio, ma **non lo visualizza ancora a schermo**.

Vediamo ora diversi casi di utilizzo:

Esempio 1 - Testo come figlio

React JSX

```react-jsx
import React from 'react';
import BigButton from './BigButton.jsx'

function App(){
    return(
        <div>
            <BigButton>
                  I am a child of BigButton.
            </BigButton>
        </div>
    )
}
export default App
```

Qui il testo `"I am a child of BigButton."` è il **figlio del componente**. Quando React esegue `BigButton`, l'oggetto `props` sarà:

JavaScript

```javascript
{
  children: "I am a child of BigButton."
}
```

Nel log apparirà quindi la stringa:

CSS

```
I am a child of BigButton.
```

Il testo, però, non viene ancora mostrato nel DOM perché non abbiamo inserito `{props.children}` all'interno del `return`.

Esempio 2 - Componente figlio

React JSX

```react-jsx
import React from 'react';
import BigButton from './BigButton.jsx';
```

```
import LilButton from './LilButton.jsx'

function App(){
    return(
        <div>
            <BigButton>
                  <LilButton />
            </BigButton>

        </div>
    )
}
export default App
```

In questo caso il figlio non è del testo, ma **un altro componente React**. React passerà l'intero elemento `<LilButton />` come valore di `props.children`.

Nel log vedremo un oggetto simile a:

JS JavaScript

```
{
  $$typeof: Symbol(react.element),
  type: LilButton,
  props: {},
   ...
}
```

Esempio 3 - Nessun figlio

React JSX

```
import React from 'react';
import BigButton from './BigButton.jsx';


function App(){
    return(
        <div>
            <BigButton />
        </div>
    )
}
```

```
export default App
```

In questo caso `props.children` non esiste, quindi React imposta:

JavaScript

```
props.children === undefined
```

Inserire le props figlie nel rendering

Per mostrare a schermo i contenuti figli, basta inserire `{props.children}` nel `return` del componente.
Ecco un esempio pratico con una lista:

React JSX

```
const List = (props) => {
  let titleText = `Favorite ${props.type}`;
  if (props.children instanceof Array) {
    titleText += 's';
  }

  return (
    <div>
      <h1>{titleText}</h1>
      <ul>{props.children}</ul>
    </div>
  );
}
```

In questo esempio:

- `props.type` viene usato per personalizzare il titolo.
- `{props.children}` viene inserito all'interno del tag `<ul>` per mostrare tutti gli elementi figli passati da `App.jsx`.

Usare lo spread operator con le props

Nella maggior parte dei casi, un componente riceve **più di una prop**.
Per esempio, nel nostro componente potremmo passare sia il `nome` che il `cognome` come attributi separati:

React JSX

```
import React from 'react';
import User from './User.jsx';


function App(){
    return(
        <div>
            <User name = 'Mattia' cognome = 'Rossi' />
        </div>
    )
}
export default App
```

Oppure, possiamo passare **tutte le props insieme** utilizzando lo **Spread Operator (`...`)**, che “espande” automaticamente un oggetto in singole proprietà:

React JSX

```
@userInfo.jsx

const userInfo = {
    id: 0,
    name: "Mattia",
    surname: "Rossi",
    eta : 44
};
export {userInfo}

@User.jsx
import {userInfo} from './userInfo.js'
const User = (props) => {
    return (
    <div>
      <h1>{props.name} {props.surname}</h1>
      <p>ID: {props.id}</p>
      <p>Età: {props.eta}</p>
    </div>
  );
}

@App.jsx
import React from 'react';
import {userInfo} from './userInfo.js';
import User from './User.jsx'
```

```
27
28
29  function App(){
30      return(
31          <div>
32              <User {...userInfo} />
33          </div>
34      )
35  }
36  export default App
```

Spiegazione:

- `userInfo` **non è un componente:**
  ma **un oggetto JavaScript definito in un file separato** (`userInfo.jsx` o meglio ancora `userInfo.js`).

> **Info**
>
> È un **modulo** che **esporta un oggetto** contenente alcune informazioni (id, nome, cognome, età).

- in `App.jsx` , viene **importato e "spacchettato" (spread)** come props nel componente `User` :

React JSX

```
1   import {userInfo} from './userInfo.jsx';
2   import User from './User.jsx'
3
4   function App(){
5       return(
6           <div>
7               <User {...userInfo} />
8           </div>
9       )
10  }
11  export default App
```

Quindi :

- `userInfo` è un **oggetto indipendente**, definito in un **file esterno**.
  Non appartiene al componente `User`, ma gli **fornisce i dati**.
- Quando si scrive: `<User {...userInfo} />`
  Si sta usando lo **Spread Operator (`...`)** per "espandere" l'oggetto in singole props.
  React quindi lo interpreta come se avessi scritto:

React JSX

```
<User id={0} name="Mattia" surname="Rossi" eta={44} />
```

In questo modo React riceverà le stesse props, ma in modo più compatto e leggibile.

Dare valori di default alle props

Spesso, quando si progetta un componente React, ci si trova nella situazione in cui esso **si aspetta di ricevere una o più props**.
Tuttavia, può capitare che tali props **non vengano passate** dal componente genitore.
In questi casi, se non si gestisce il problema, il componente potrebbe **mostrare contenuti vuoti** o addirittura **generare errori**.

Per evitare questo comportamento, è buona pratica **assegnare valori di default alle props**.
In questo modo, il componente avrà sempre un comportamento prevedibile, anche in assenza di dati.

Esempio di partenza

Supponiamo di avere un componente `Button3` che si aspetta una prop chiamata `text`:

React JSX

```
function Button3(props) {
  return (
    <button>{props.text}</button>
  );
}
```

Se la utilizzassimo così:

React JSX

```
<Button3 text="Clicca qui!" />
```

verrà mostrato il bottone con il testo *“Clicca qui!”*.
Ma se scriviamo semplicemente:

React JSX

```
<Button3 />
```

il bottone verrà renderizzato **vuoto**, poiché non è stato passato alcun valore a `props.text`.

Assegnare valori di default

Per evitare questo, possiamo **definire un valore predefinito** per la prop `text`.
In React esistono **tre metodi principali** per farlo.

1. **Metodo classico – Proprietà statica `defaultProps`:**

React JSX

```
function Button3(props) {
  return (
    <button>{props.text}</button>
  );
}

Button3.defaultProps = {
  text: 'This is default text',
};
```

In questo caso, `defaultProps` è una **proprietà statica del componente**.
Se la prop `text` non viene fornita, React userà automaticamente il valore `"This is default text"`.

> **Info**
>
> **Questo metodo era molto comune nelle versioni precedenti di React, ma oggi è meno usato nei Functional Components in favore dei metodi seguenti.**

2. **Metodo moderno – Default nella destrutturazione delle props:**

React JSX

```
function Button3({ text = 'This is default text' }) {
  return <button>{text}</button>;
}
```

Qui il valore di default viene definito **direttamente nella destrutturazione** delle props.
È una sintassi molto pulita e leggibile, e rappresenta **il metodo più usato** nei componenti funzionali moderni.

Cosa succede?

Se `text` non viene passato, verrà automaticamente impostato su `"This is default text"`.
Se invece `text` viene fornito, il valore passato dal genitore **sovrascriverà** quello di default.

3. **Metodo nel corpo della funzione:**

React JSX

```
function Button3(props) {
  const { text = 'This is default text' } = props;
  return <button>{text}</button>;
}
```

Qui si imposta il valore di default **all'interno del corpo della funzione**, tramite una destrutturazione con assegnazione di default.
Il risultato finale è lo stesso, ma questo approccio è utile quando si vogliono eseguire **altre operazioni logiche** prima del render.

Esempio pratico

React JSX

```
function Button3({ text = 'Default Text of Big Button' }) {
return (
  <button>{text}</button>
```

```
);
}
```

Se il componente in `App.jsx` viene usato cosi:

React JSX

```
<Button3 />
```

verrà visualizzato un bottone con il testo:

Wolfram language

```
Default Text of Big Button
```

Mentre se lo si utilizza cosi:

```
<Button3 text="Invia modulo" />
```

Il valore di defualt verrà sovrascritto e il bottone mostrerà:

nginx

```
Invia modulo

```

Lavorare con gli array in React

Quando si lavora con **collezioni di dati**, come array di oggetti o liste di elementi, è molto comune volerli **mostrare dinamicamente** all'interno di un componente React.
Per farlo, si utilizza il **metodo** `.map()`, uno strumento fondamentale in JavaScript e particolarmente utile in React.

Cos'è e come funziona `.map()`

Il metodo `.map()` :

- **è un metodo nativo degli array in JavaScript.**
- **Permette di iterare su tutti gli elementi di un array, eseguendo una funzione su ciascuno di essi, e ritorna un nuovo array contenente i**

risultati.

In altre parole, `.map()` ci consente di trasformare un array in un altro array, **senza modificare quello originale**.

Ecco un esempio generico:

JS JavaScript

```javascript
const numeri = [1, 2, 3];
const raddoppiati = numeri.map(numero => numero * 2);

console.log(raddoppiati); // Output: [2, 4, 6]
```

In questo caso, `.map()` ha iterato sull'array `numeri` e per ogni elemento ha applicato la funzione `numero * 2`, creando così un nuovo array `raddoppiati`.

Uso di `.map()` in React

In React, il metodo `.map()` è **estremamente utile** per **renderizzare liste di elementi** in modo dinamico.

Immaginiamo di avere un array di persone:

JS JavaScript

```javascript
const persone = [
  { id: 1, nome: "Luca" },
  { id: 2, nome: "Giulia" },
  { id: 3, nome: "Mattia" }
];
```

Se vogliamo visualizzare ciascun nome come elemento `<li>`, possiamo scrivere:

React JSX

```jsx
function ListaPersone() {
  return (
    <ul>
      {persone.map((persona) => (
        <li>{persona.nome}</li>
      ))}
    </ul>
  );
```

```
}
```

Ogni elemento dell'array `persone` viene elaborato a turno, e React renderizza una `<li>` per ciascuno.

⚠️ Il problema del "key" attribute

Se eseguiamo il codice precedente, noteremo un **warning nella console del browser**.
React ci avviserà che ogni elemento di una lista **deve avere un attributo** `key` **univoco**.

Questo accade perché, internamente, React ha bisogno di **identificare in modo univoco ogni elemento** della lista per poter gestire correttamente eventuali aggiornamenti, modifiche o rimozioni.
Senza la prop `key`, React non può ottimizzare il rendering e può confondere gli elementi della lista durante gli aggiornamenti del DOM virtuale.

Come assegnare l'attributo `key`

Ci sono due modi per assegnare correttamente la `key`:

1. Usare l'indice dell'array:
   Il metodo `.map()` accetta un **secondo parametro**, chiamato *index*, che rappresenta la posizione dell'elemento all'interno dell'array.

Possiamo sfruttarlo così:

React JSX

```jsx
<ul>
  {persone.map((persona, index) => (
    <li key={index}>{persona.nome}</li>
  ))}
</ul>

```

> Questo metodo funziona, ma **non è sempre consigliato**:
> se l'array cambia dinamicamente (es. elementi rimossi, riordinati o aggiunti), gli indici possono cambiare, causando problemi al rendering di React.

2. **Usare un identificatore univoco:**
   Il modo **più corretto** è quello di **aggiungere un campo univoco** (come `id`)

all’interno dell’oggetto e usare quello come chiave:

React JSX

```react-jsx
<ul>
  {persone.map((persona) => (
    <li key={persona.id}>{persona.nome}</li>
  ))}
</ul>

```

In questo modo, anche se l’array cambia, React sarà sempre in grado di distinguere correttamente gli elementi grazie al valore stabile di `id`.

Consiglio pratico: gestire i dati in file separati

Per mantenere il codice più pulito e leggibile, soprattutto nei progetti didattici dove non è presente un database, è una buona pratica **memorizzare i dati in un file** `.js` **esterno** ed **importarli** all’interno dei componenti che ne hanno bisogno.

Ad esempio:

JavaScript

```javascript
// datiPersone.js
const persone = [
  { id: 1, nome: "Luca" },
  { id: 2, nome: "Giulia" },
  { id: 3, nome: "Mattia" }
];

export { persone };

```

E poi nel componente:

React JSX

```react-jsx
import { persone } from './datiPersone.js';

function ListaPersone() {
      return (
```

```
5          <ul>
6            {persone.map((persona) => (
7                  <li key={persona.id}>{persona.nome}</li>
8            ))}
9          </ul>
10       );
11   }
```

Gli event handler nei componenti

Quindi come abbiamo visto in React è molto comune passare una funzione come *prop* a un componente figlio, in modo che quest'ultimo possa **gestire eventi** (come un clic, un passaggio del mouse o un input dell'utente).

Questa funzione prende il nome di **Event Handler** e, come vedremo, può essere definita nel componente genitore e poi passata come prop al componente figlio.

Esempio base: funzione event handler passata come prop

React JSX

```
1
2   @Button.jsx
3   const Button = (props) =>{
4       // Restitusco un elemento button a cui aggiungo un onClick
5       // e come valore passo la proprietà talk definita nel
    componente padre Talker in props.jsx
6        // A cui accedo tramite dot notation
7       return (
8           <button onClick = {props.talk}>
9               Click me!
10          </button>
11      );
12  }
13  export default Button;
14
15  @Talker.jsx
16  import Button from './Button.jsx'
17  const Talker = () => {
18    // 1️⃣ Definisco una funzione event handler
19    function talk() {
20      // Creo una variabile vuota dove accumulerò testo
```

```
21      let speech = '';
22      for (let i = 0; i < 10000; i++) {
23        speech += 'blah';
24      }
25      // Mostro il risultato in un alert
26      alert(speech);
27    }
28
29    // 2️⃣ Passo la funzione come prop al componente Button
30    return <Button talk={talk} />;
31  }
32  export default Talker
33
```

In questo esempio:

- `Talker` **è il componente padre, che definisce la funzione** `talk()`**.**
- **La funzione viene poi passata come prop al componente figlio** `<Button />`**.**
- **Nel componente figlio** `Button` **l'evento viene gestito tramite l'event listeners** `onClick`**; quindi ogni volta che l'utente clicca sul pulsante, React esegue la funzione** `talk()` **passata tramite la prop.**

> ✓ **In questo modo, la logica dell'evento (** `talk` **) rimane nel componente genitore, mentre il componente figlio (** `Button` **) si occupa solo della parte grafica (il rendering del bottone).**

Naming convention : `handleEvent`, `onEvent`, e `props.event`

Quando si lavora con Event Handler in React, esistono due nomi da scegliere:

1. **Il nome della funzione handler, definita nel componente padre.**
2. **Il nome della prop, con cui si passa la funzione al componente figlio.**

Nome della funzione handler

La convenzione in React vuole che le funzioni che gestiscono eventi abbiano un nome composto da:

```nginx
1  handle + NomeEvento
```

Esempi:

- `handleClick()` → per gestire un click del mouse
- `handleHover()` → per gestire il passaggio del cursore.

React JSX

```react-jsx
import Child from './Child.jsx';
const My Component = ()=> {
    function handleHover(){
        alert ('Evento hover intercettato!')
    }
    return <Child onHover = {handleOver} />
}

```

Nome della prop (nel JSX)

La convenzione per le props che rappresentano eventi è usare il prefisso:

nginx

```nginx
on + NomeEvento
```

Esempio

- `onClick={handleClick}`
- `onHover={handleHover}`

> ✓ **Queste convenzioni non sono obbligatorie, ma altamente consigliate, perché rendono il codice più leggibile e coerente con la sintassi JSX di React.**

Esempio completo : collegare l'handler al bottone

React JSX

```react-jsx
import Button2 from './Button2.jsx'
```

```
const Speecher = () => {
  // Definisco una funzione handler chiamata handleClick
  function handleClick() {
    let talk = '';
    for (let i = 0; i < 10000; i++) {
      talk += 'blah';
    }
    alert(talk);
  }

  // Ritorno un componente Button a cui passo la funzione come prop
  return <Button2 onClick={handleClick} />;
}
```

⚠Attenzione:

Nel codice qui sopra, `onClick` **non crea realmente un event listener**:

- è **solo il nome di una prop** passata a un componente React personalizzato (`Button2`).

Gli event listener reali vengono creati solo sugli elementi JSX nativi, come `<button>`, `<input>`, `<div>` ecc.

Quando usiamo `onClick` su un componente React, è soltanto un canale di comunicazione tra il genitore e il figlio.

Quindi:

React JSX

```
// Button2.jsx
const Button2 = ({ onClick }) => {
  return (
    <button onClick={onClick}>
      Parla!
    </button>
  );
}
export default Button2;

```

In questo caso l'attributo `onClick` è veramente collegato a un **event listener**, perché è applicato a un elemento HTML (`<button>`).

Event Listeners vs. Event Handler

1. Gli event Listeners:
   Un event listener ("ascoltatore di eventi") è un meccanismo del browser (parte del DOM) che "ascolta" un determinato evento — ad esempio un click, un input da tastiera o il passaggio del mouse.

In JavaScript puro (senza React) puoi aggiungere un listener così:

JS JavaScript

```javascript
const button = document.querySelector('button');
button.addEventListener('click', function() {
  alert('Hai cliccato!');
});
```

Significa : "browser, quando l'utente clicca su questo bottone, esegui questa funzione".

Quindi:

- L'**event listener** è la **connessione tecnica** tra il DOM e la funzione che reagirà all'evento.
- Il browser lo registra e lo attiva automaticamente al verificarsi dell'evento.

2. Gli event handler:
   Un event handler ("gestore di evento") è semplicemente la funzione che gestisce l'evento — cioè il codice da eseguire quando l'evento si verifica.

Nel codice sopra, la funzione anonima passata a `addEventListener` è proprio **l'handler**:

JS JavaScript

```javascript
function() {
```

```
  alert('Hai cliccato!');
}
```

In Sintesi

- L'**event listener: "ascolta" l'evento**.
- **L'event handler: "gestisce" cosa accade quando l'evento viene ascoltato.**

Come funziona in React

In React, **non si usa direttamente** `addEventListener`, perché:

- React gestisce internamente gli eventi attraverso un sistema chiamato **Synthetic Events** (eventi sintetici, astratti dal DOM reale).
  In pratica quando si scrive:

React JSX

```
<button onClick={handleClick}>Clicca qui</button>
```

Si sta dicendo:

> "React, quando questo bottone viene cliccato, esegui `handleClick`".
> Qui:

- `onClick` → **è un attributo JSX speciale che React traduce internamente in un event listener.**
- `handleClick` → **è la funzione handler che React chiamerà quando l'evento avviene.**

Esempio completo:

React JSX

```
function App() {
    function handleClick() {
        alert("Hai cliccato il bottone!");
    }
    return <button onClick={handleClick}>Clicca</button>;
}
```

React, dietro le quinte:

1. Registra **un unico event listener globale** sul `document`.
2. Quando avviene un evento (es. click), React intercetta l'evento e **lo propaga virtualmente** all'elemento JSX interessato.
3. Quando trova l'elemento `<button>` con `onClick={handleClick}`, **invoca la funzione handler**.

Oggetto e evento dell'oggetto

Cos'è un oggetto in JavaScript

In JavaScript, come in Python, quando si crea un oggetto (che può essere paragonato a un dizionario in Python o a un oggetto letterale in JS), **la variabile non contiene direttamente l'oggetto, ma un riferimento alla zona di memoria in cui l'oggetto è memorizzato.**

In altre parole:

- **la variabile è un'etichetta,**

- l'oggetto vero e proprio si trova nella RAM,

- e la variabile punta all'indirizzo di memoria dove risiede tale oggetto.

Detto in sintesi:

> **Un oggetto è un valore che risiede in una cella della memoria RAM e la variabile punta all'indirizzo di memoria di quell'oggetto.**
> **Esempio in Python**

Python

```python
a:dict[str, str] = {"nome": "Marco"}
b:dict[str, str] = a  # b punta allo stesso oggetto di a
b["nome"] = "Luca"
print(a["nome"])  # Output: "Luca"
```

Spiegazione:

- `a` non *contiene* il dizionario, ma un **riferimento** all'oggetto dizionario in memoria.
- Quando fai `b = a`, **non viene creato un nuovo dizionario** — `b` riceve solo lo stesso riferimento.
- Modificando `b`, in realtà modifichi **l'unico oggetto condiviso** tra `a` e `b`.

Esempio equivalente in JavaScript

JS JavaScript

```javascript
let a = { nome: "Marco" };
let b = a;      // b punta allo stesso oggetto di a
b.nome = "Luca";
console.log(a.nome);  // Output: "Luca"
```

Esattamente lo stesso comportamento di Python.
In JavaScript anche gli **oggetti**, **array** e **funzioni** vengono gestiti per **riferimento in memoria**.

Tipi primitivi vs tipi per riferimento

In JavaScript esistono due grandi categorie di tipi di dati:

- **Primitivi** (`string`, `number`, `boolean`, `null`, `undefined`, `symbol`, `bigint`):
  - vengono **gestiti per valore**, cioè la variabile **contiene direttamente il valore**.
- **Per riferimento** (`object`, `array`, `function`):
  - vengono **gestiti per riferimento**, quindi la variabile **contiene l'indirizzo di memoria** dell'oggetto.
    Esempio con valore primitivo

JS JavaScript

```javascript
let x = 10;
let y = x;
y = 20;
console.log(x); // 10
```

Qui `x` e `y` sono indipendenti, perché i **numeri** (come anche stringhe e booleani) sono **tipi primitivi** e vengono copiati **per valore**.

Esempio con oggetto (riferimento)

JS JavaScript

```javascript
let obj1 = { name: "Marco" };
let obj2 = obj1;
obj2.name = "Luca";
console.log(obj1.name); // "Luca"
```

Qui invece, `obj1` e `obj2` condividono **lo stesso riferimento in memoria**:

- modificare `obj2` significa modificare l'oggetto puntato anche da `obj1`.

In sintesi

Un oggetto è una struttura dati complessa che consente di rappresentare e organizzare informazioni correlate in forma di coppie chiave–valore.

In JavaScript (come in Python o Java), un oggetto può contenere:

- **dati (proprietà o attributi),**

- funzionalità (metodi o funzioni associate).

Dal punto di vista della memoria, un oggetto **non è memorizzato direttamente nella variabile**,
ma in una sezione dedicata della **RAM**.
👉 La variabile contiene soltanto un **riferimento (puntatore)** all'indirizzo di memoria dove risiede l'oggetto.

> **In sintesi**
>
> Un oggetto è un'entità in memoria che rappresenta un insieme di dati e comportamenti associati,
> accessibili e modificabili tramite un riferimento.

Cos è un evento in JavaScript

Un evento è qualsiasi azione o accadimento che avviene durante l'esecuzione di un programma e che può essere rilevato e gestito dal

software.

Nel contesto del **browser** e di **React**, un evento rappresenta un'interazione dell'utente o del sistema, come ad esempio:

- **un clic del mouse (`click`),**
- **la pressione di un tasto (`keydown`)**,
- **la digitazione in un campo di input (`input` o `change`)**,
- **il caricamento della pagina (`load`)**,
- **o un aggiornamento di stato interno**.

Ogni volta che un evento si verifica, il browser **crea un oggetto evento** (`event object`) che descrive in dettaglio ciò che è accaduto — ad esempio:

- quale elemento lo ha generato,
- quale tasto è stato premuto,
- che valore è stato inserito, ecc.

Questo oggetto viene poi passato automaticamente alla funzione che gestisce l'evento, chiamata event handler.

> **In sintesi**
>
> **Un evento è un segnale che indica che qualcosa è accaduto (un'azione dell'utente o del sistema) e che può essere gestito da una funzione specifica chiamata *event handler*.**

Esempio in React:

React JSX

```react-jsx
const Bottone = () => {
    const handleClick = (event) => {
        console.log("Hai cliccato!", event); // 'event' è l'oggetto evento
      };

    return <button onClick={handleClick}>Cliccami</button>;
    }
```

Qui:

- `onClick` è un **listener**: “ascolta” l’evento `click`;
- `handleClick` è l’**event handler**: la funzione che viene eseguita quando l’evento si verifica;
- `event` è l’**oggetto evento** creato automaticamente da React/Browser, e contiene informazioni come `event.target`, `event.type`, `event.timeStamp`, ecc.